

1. Visão Geral do Sistema

1.1. Nome e propósito

COEPP – Sistema web para gestão de pacientes, estagiários e agendamentos (triagem, acompanhamento e terapia), com lista de prioridades, calendário, relatórios de atendimentos e controle de acesso.

1.2. Arquitetura

Linguagem/stack: PHP (mysqli), HTML/CSS/JS, Chart.js (gráficos), FullCalendar (calendário – previsto).

Organização por módulos:

config/ (bootstrap, conexão, helpers).

includes/ (layout, navbar, guard de autenticação, modal de info).

auth/ (login/logout).

dashboard/ (módulos de Pacientes, Estagiários, Agendamentos, Calendário, Relatórios).

assets/ (CSS).

Segurança:

Prepared Statements (SQL).

XSS: htmlspecialchars nas saídas.

Auth Guard: bloqueia acesso sem sessão.

.htaccess: desabilita index de diretório, ativa rewrite.

2. Configuração e Bootstrap (/config)

2.1. config/init.php

Função: bootstrap global do app.

Principais responsabilidades:

Inicia sessão (session_start()) quando necessário).

Define constantes: ROOT (raiz física do projeto), APP_URL (URL base).

Carrega conexão MySQLi (armazenada em \$GLOBALS['mysqli']).

Helper de URL: url(\$path) para montar caminhos absolutos com base em APP_URL.

(Em dev, pode habilitar mysqli_report para facilitar debugging).

Entradas/Saídas:

Entrada: nenhuma (invocado em quase todos os scripts).

Saída: variáveis globais/constantes preparadas.

Observações:

Centraliza a única instância de conexão com o banco.

Garante charset utf8mb4 (em geral em cada script que manipula DB).

2.2. config/db.php (ou dp.php)

Função: armazena credenciais do banco (host, dbname, user, pass).

Observações:

Ideal manter fora do versionamento (ou com .example).

Usar utf8mb4.

3. Autenticação (/auth)

3.1. auth/login.php

Função: tela/processo de login.

Fluxo:

GET: exibe formulário (email, senha).

POST: valida credenciais via DB, password_verify.

Se OK, define \$_SESSION['user_id'], \$_SESSION['user_email'] e redireciona para dashboard/index.php.

Validações:

Campos obrigatórios.

Erro amigável caso usuário/senha inválidos.

3.2. auth/logout.php

Função: encerrar sessão e redirecionar para login.

Fluxo:

session_unset(), session_destroy().

header('Location: auth/login.php').

4. Includes / Layout (/includes)

4.1. includes/auth_guard.php

Função: middleware de autenticação.

Regra:

Considera logado se existir `$_SESSION['user_id']` ou (legado) `$_SESSION['email']`.

Senão, Location: `/auth/login.php`.

Observações:

Usado no topo de praticamente todas as páginas do dashboard.

4.2. includes/header.php

Função: cabeçalho HTML + topbar, sidebar e links ativos.

Destaques:

Calcula `APP_URL` quando não definido.

Helpers:

`is_active($paths)` para setar classe “is-active”.

Define URLs globais (dashboard, pacientes, preferenciais, estagiários, atendidos, calendário, dúvidas).

Renderiza topbar (logo, título, e-mail do usuário) e sidebar (links).

Inicia `<main class="content">`.

Observações:

Exibe e-mail do usuário a partir de `$_SESSION` (várias chaves de fallback).

CSS inline mínimo para layout (complementado por `/assets/css/app.css`).

4.3. includes/navbar.php

Função: alternativa de sidebar com ícones SVG inline e botão “Compactar”.

Destaques:

A função `item()` encapsula o link com estado ativo.

`nav_active()` determina ativo pelos trechos da URL.

Botão “Compactar” grava estado em `localStorage`.

4.4. includes/info_modal.php

Função: drawer lateral reutilizável para mostrar ficha de Paciente/Estagiário.

JS:

openInfo(tipo, id) busca (fetch) o endpoint correto:

Paciente: dashboard/pacientes/api_show.php?id=...

Estagiário: dashboard/estagiarios/api_show.php?id=...

Renderização em pares chave-valor.

4.5. includes/footer.php

Função: fecha <main> e injeta <footer>. Inclui info_modal.php.

Observação:

Mostra © COEPP / CodeLink — Período de testes.

5. Recursos Estáticos (/assets/css)

5.1. assets/css/app.css

Função: estilos globais do sistema.

Conteúdo (macro):

Layout de topbar, sidebar, content.

Componentes: .card, .table, .btn, paleta #0a4ea1.

Sugestão: migrar CSS inline das telas para cá aos poucos.

6. Página Inicial / KPIs (/dashboard/index.php)

6.1. dashboard/index.php

Função: exibe KPIs + gráfico (Chart.js) + acessos rápidos.

KPIs calculados:

Consultas de Hoje (contagem em agendamentos por data = hoje).

Pacientes na Lista de Prioridade (preferencial = 1 ou caso_preferencial = 'Sim').

Pacientes sem Prioridade (total - prioritários).

Quantidade de Cadastros (total em pacientes).

Gráfico:

Série Atendimentos realizados no mês (contagem por dia de agendamentos no mês/ano selecionados).

Filtros: mês e ano (selects).

Chart.js via CDN.

Técnicas:

first_existing_col() (consulta INFORMATION_SCHEMA) para detectar colunas.

prepared statements para contagens específicas.

Observações:

Silêncio em exceptions para não quebrar o dashboard.

“Acessos rápidos” para calendário, pacientes, estagiários, prioridade e relatório.

7. Módulo: Pacientes (/dashboard/pacientes)

7.1. pacientes.php

Função: listagem e busca de pacientes.

Busca por: nº prontuário, nome, CPF (LIKE com wildcard).

Tabela: nº prontuário, nome, data de nascimento (formatada), telefone, CPF (formatado), email.

Ações:

 Informações (abre modal que chama api_show.php).

 Editar (editar_paciente.php?id=X).

 Excluir (excluir_paciente.php?id=X – confirmação JS).

Observações:

Usa formataadores locais (telefone, CPF, data).

Ordena por CAST(numero_prontuario AS UNSIGNED).

7.2. cadastrar_paciente.php

Função: formulário de novo cadastro.

Campos obrigatórios:

nome, telefone, CPF, endereço, e-mail, data de nascimento, estuda FSA (select), tipo de serviço.

Preferencial (checkbox) + observação (opcional).

RA obrigatório se estuda FSA = “Sim”.

UX/JS:

Máscaras: CPF, telefone, RA (6 dígitos).

Validações JS no submit (CPF 11 dígitos, telefone 10/11, RA 6 quando necessário).

Mostra nº prontuário estimado (só para exibição).

POST → processa_paciente.php.

7.3. processa_paciente.php

Função: INSERT de paciente.

Validações no servidor:

Obrigatórios (mesmos da UI).

CPF 11 dígitos → formata para 000.000.000-00.

estuda_fsa: aceita "1/0" ou "Sim/Não" → normaliza p/ 0/1.

Se não estuda FSA → ra = NULL.

Unicidade de CPF e e-mail (consultas preparadas).

Nº prontuário:

SELECT COALESCE(MAX(numero_prontuario),0) + 1 → evita duplicidade.

Insert:

Campos: numero_prontuario, nome, data_nascimento, telefone, cpf, endereco, preferencial, preferencial_obs, estuda_fsa, ra, encaminhamento, tipo_servico, email.

Saída: redireciona pacientes.php?ok=1 ou erro amigável.

7.4. editar_paciente.php

Função: formulário de edição.

Destaques:

Pré-carrega dados (SELECT * WHERE id=?).

Normaliza preferencial/estuda_fsa para 0/1.

Mostra RA somente quando estuda_fsa=1.

POST → atualizar_paciente.php.

7.5. atualizar_paciente.php

Função: UPDATE com validações.

Validações:

Obrigatórios.

CPF 11 dígitos → formatação.

estuda_fsa: normaliza. Se 0 → RA NULL.

Unicidade de CPF e e-mail ignorando o próprio ID.

Update: atualiza todos os campos relevantes.

Saída: pacientes.php?ok=1 ou erro.

7.6. excluir_paciente.php

Função: DELETE do paciente com checagem de vínculos.

Fluxo:

Verifica se paciente existe (idempotência: se não, retorna ok).

Conta agendamentos vinculados (agendamentos.paciente_id ≥ hoje; ignora cancelado se existir coluna status).

Se houver, bloqueia exclusão e instrui acessar Agendamentos (passa redir com sugestão).

Tenta deletar (se erro 1451/FK → mensagem apropriada).

7.7. api_show.php

Função: endpoint JSON para montar HTML do modal de informações.

Dados formatados:

CPF, telefone, data.

“Estuda FSA?” com RA se aplicável.

Badge Preferencial, se marcado.

Created/updated (formatados) – se existirem.

Saída: { ok:true, html: "<div>...</div>" } ou erro.

7.8. lista_prioridade.php

Função: visualização comparativa de:

Pacientes com Prioridade (preferencial/caso_preferencial).

Demais Pacientes.

Exclusão por agendamento ativo:

Exclui pacientes que já têm agendamento futuro (≥ hoje)

Se agendamentos.status existir, ignora “cancelado”.

Colunas dinâmicas:

Verifica existência de preferencial (TINYINT 0/1) ou caso_preferencial ('Sim'/'Não').

Se nenhuma existir, instruções de ALTER TABLE.

Ação: botão Agendar → `dashboard/agendamentos/novo.php?paciente_id=ID`.

7.9. preferenciais.php

Função: lista somente os preferenciais (campo preferencial=1).

Busca: nº prontuário, nome, CPF.

Tabela: dados básicos, com formatações.

8. Módulo: Estagiários (/dashboard/estagiarios)

8.1. estagiarios.php

Função: lista geral de estagiários com busca.

Busca: matrícula, nome, supervisor (LIKE).

Colunas: matrícula, nome, semestre, supervisor, tipo de serviço, telefone, e-mail, disponibilidade.

Disponibilidade (exibição):

Converte JSON { dia: ["HH:MM", ...] } em blocos por dia.

Mapeia “terça/sábado” e variantes sem acento.

Formatação de telefone.

Ações:

 Informações (modal → `api_show.php`).

 Editar.

 Excluir (com confirmação).

UX extra: barra de rolagem superior sincronizada com a tabela grande.

8.2. cadastrar_estagiario.php

Função: formulário de novo estagiário.

Campos: nome, matrícula, telefone, email, semestre (4º-8º), supervisor, tipo_serviço (Triagem/Acompanhamento/Terapia), disponibilidade por grade.

UX disponibilidade:

Presets: Nenhum, Manhã (8–12), Tarde (13–17), Noite (18–20), Todos (8–20).

Copiar Segunda para todos.

“Editar horários” por dia (expande grade de checkboxes).

POST → salvar_estagiario.php.

8.3. salvar_estagiario.php

Função: INSERT de estagiário.

Validações:

Obrigatórios: nome, matrícula, telefone, email, supervisor.

tipo_servico ∈ {Triagem, Acompanhamento, Terapia}.

Telefone (10/11 dígitos), e-mail válido.

Semestre ∈ {'4','5','6','7','8'}.

Disponibilidade: pelo menos um horário; normaliza JSON.

Insert: com disponibilidade em JSON (dias ordenados e únicos).

Saída: estagiarios.php?ok=1 ou erro.

8.4. editar_estagiario.php

Função: formulário de edição (pré-carrega).

Destaques:

Hidrata JSON da disponibilidade nos checkboxes.

Preserva o mesmo UX da criação (presets, copiar, limpar).

POST → atualizar_estagiario.php.

8.5. atualizar_estagiario.php

Função: UPDATE com validações idênticas ao insert.

Saída: estagiarios.php?ok=1 ou erro amigável.

8.6. excluir_estagiarios.php

Função: DELETE com prepared statement.

Tratamento:

Se erro 1451 (FK), exibe mensagem “há registros relacionados”.

Caso sucesso: redireciona com ok=1.

8.7. api_show.php

Função: endpoint JSON (HTML pronto) com ficha do estagiário.

Inclui:

Nome, matrícula, semestre (4º–8º), supervisor, tipo de serviço, telefone, e-mail.

Disponibilidade “humana”.

8.8. carregar_dados_estagiario.php

Função: endpoint JSON para carregar dados específicos (supervisor, tipo_servico e disponibilidade) por ID.

Notas:

Retorna disponibilidade como objeto (ou {}).

Trata JSON inválido retornando a string original (fallback).

9. Módulo: Agendamentos (/dashboard/agendamentos)

Importante: parte desses arquivos não foi enviada em código-fonte, mas são referenciados pelas telas (lista de prioridade, dashboard, calendário e relatórios). Abaixo, a documentação reflete o padrão do projeto e os contratos já usados nas páginas existentes.

9.1. agendamentos.php (referenciado)

Função: listagem geral de agendamentos com busca (por paciente, estagiário, data e status).

Colunas esperadas:

Data, Hora, Duração/Tipo, Paciente, Estagiário, Sala (se aplicável), Status (marcado, realizado, cancelado).

Ações:

Ver/Detalhes (modal ou página).

Editar (editar.php?id=X).

Cancelar (cancelar_agendamento.php?id=X).

Excluir (excluir.php?id=X).

Confirmar/Realizar (confirmar_agendamento.php?id=X).

9.2. novo.php (referenciado)

Função: formulário de novo agendamento.

Entradas:

paciente_id (pode vir pré-preenchido da lista de prioridade).

estagiario_id, data, hora, duração, sala (se houver), tipo de serviço.

Integrações:

Consulta de disponibilidade (timeslots.php, carregar_agendamentos.php) para mostrar horários livres.

POST → salvar.php.

9.3. salvar.php (referenciado)

Função: INSERT de agendamento.

Validações:

Campos obrigatórios.

Conflito de agenda do estagiário e de sala (se aplicável).

Paciente não deve ter agendamento ativo no mesmo horário.

Saída: redireciona para agendamentos.php?ok=1.

Função: formulário de edição.

Entradas: id.

Comportamento:

Carrega dados atuais do agendamento.

Mostra slots disponíveis para a nova data/estagiário.

POST para atualizar.php.

9.5. atualizar.php (referenciado)

Função: UPDATE.

Validações: iguais às de salvar.php.

Saída: agendamentos.php?ok=1 ou erro.

9.6. excluir.php (referenciado)

Função: DELETE.

Regra sugerida:

Bloquear exclusão se status = 'realizado' (ou preferir cancelamento).

Confirmar via JS.

9.7. cancelar_agendamento.php (referenciado)

Função: seta status = 'cancelado' e persiste motivo (opcional).

Saída: agendamentos.php?ok=1.

9.8. confirmar_agendamento.php (referenciado)

Função: seta status = 'realizado' + registra horário de realização.

Saída: agendamentos.php?ok=1.

9.9. atendidos.php (referenciado)

Função: relatório de agendamentos realizados.

Filtros: intervalo de datas, estagiário, tipo de serviço.

Ações: exportar CSV (exportar_agendamentos.php).

9.10. exportar_agendamentos.php (referenciado)

Função: gera arquivo CSV/Excel segundo filtros do relatório.

9.11. carregar_agendamentos.php (referenciado)

Função: endpoint JSON com agendamentos (usado em listas “dinâmicas” e/ou calendário).

Entradas esperadas: start, end, estagiario_id (opcional), status (opcional).

Saída: array com campos básicos (id, paciente_nome, estagiario_nome, data, hora, status).

9.12. timeslots.php (referenciado)

Função: calcula horários livres de um estagiário para uma data e duração.

Regras típicas:

Interseção entre disponibilidade do estagiário (JSON) e agendamentos existentes.

Se houver salas: interseção com salas disponíveis.

Retorno: lista com hora_inicio, hora_fim (e, se houver, sala).

10. Calendário (/dashboard/calendario)

Assim como agendamentos, parte deste módulo foi referenciada, não enviada. Abaixo, contrato esperado para integração com as outras telas.

10.1. calendario.php (referenciado + já listado no dashboard/index “Acessos Rápidos”)

Função: tela com FullCalendar embarcado.

Integrações:

Events feed: events.php (JSON).

(Opcional) Disponibilidades como “background events” (ex.: availability.php).

Filtros comuns: estagiário, tipo de serviço, status.

10.2. events.php (referenciado)

Função: fornece eventos em formato compatível com FullCalendar.

Entradas: start, end, estagiario_id (opcional), status (opcional).

Saída: [{ id, title, start, end, extendedProps: { paciente, estagiario, status } }].

10.3. calendario_feed.php (referenciado)

Função: variante/legado do feed de eventos (pode ser igual a events.php com outro contrato).

11. Raiz do Projeto (/)

11.1. .htaccess

Função: regras de Apache para segurança e roteamento.

Configurações:

Options -Indexes -MultiViews.

DirectoryIndex index.php.

mod_rewrite habilitado (RewriteBase /coepp/).

Regras para mapear pastas diretamente para seus arquivos .php (estagiários, pacientes, usuários, agendamentos, calendário, dúvidas).

Charset default utf-8.

11.2. index.php

Função: root router.

Fluxo:

Se não logado (`$_SESSION['user_id']` vazio) → `auth/login.php`.

Se logado → `dashboard/index.php`.

12. Segurança e Boas Práticas

12.1. Segurança aplicada

Autenticação: `includes/auth_guard.php`.

SQL Injection: prepared statements (`bind_param`).

XSS: `htmlspecialchars` em saídas (e.g., `helpers e()`).

`.htaccess`: protege diretórios (sem `index`), `rewrite base`.

12.2. Recomendações adicionais

CSRF tokens em formulários (hidden token por sessão).

Logs de auditoria para ações críticas (cadastro, edição, exclusão, status).

Rate limit em login.

Campos ENUM padronizados (e.g., `agendamentos.status ∈ {'marcado','realizado','cancelado'}`).

Centralizar máscaras e validações JS (telefone, CPF) em um arquivo JS único.

Mover CSS inline para `assets/css/app.css` gradualmente.

13. Fluxos Principais

13.1. Cadastro de Paciente

`cadastrar_paciente.php` → validações JS.

POST → `processa_paciente.php`

valida, gera nº prontuário, INSERT.

`pacientes.php?ok=1`.

13.2. Edição de Paciente

`editar_paciente.php?id=X` (pré-carrega).

POST → atualizar_paciente.php

valida, UPDATE, unicidade ignorando próprio id.

pacientes.php?ok=1.

13.3. Exclusão de Paciente

excluir_paciente.php?id=X.

Bloqueia se houver agendamentos futuros (e não cancelados).

Caso contrário, DELETE.

13.4. Cadastro de Estagiário

cadastrar_estagiario.php (grade de disponibilidade).

POST → salvar_estagiario.php (normaliza JSON, valida).

estagiarios.php?ok=1.

13.5. Lista de Prioridade

lista_prioridade.php detecta coluna (preferencial ou caso_preferencial).

Exclui pacientes com agendamento futuro.

Oferece “Agendar” (agendamentos/novo.php?paciente_id=...).

13.6. Dashboard / KPIs

Conta registros (pacientes, prioritários).

Conta consultas do dia (agendamentos).

Gráfico atendimentos do mês com Chart.js.

14. Esquema de Banco (resumo funcional)

Baseado no uso observado (ajuste conforme seu schema real).

14.1. pacientes

id (PK, AI),

numero_prontuario (INT/UNIQUE),

nome, data_nascimento (DATE), telefone, cpf (FORMATO 000.000.000-00),

endereco, email (UNIQUE),

preferencial (TINYINT 0/1) ou caso_preferencial ('Sim'/'Não'),
preferencial_obs,
estuda_fsa (TINYINT 0/1), ra (6 dígitos ou NULL),
encaminhamento, tipo_servico,
created_at, updated_at.

14.2. estagiarios

id (PK, AI),
nome, matricula, telefone, email,
semestre (VARCHAR '4'..'8' — compatível com ENUM),
supervisor, tipo_servico,
disponibilidade (JSON { dia: ["HH:MM", ...] }).

14.3. agendamentos

id (PK, AI),
paciente_id (FK → pacientes),
estagiario_id (FK → estagiarios),
data (DATE), hora_inicio (TIME), hora_fim (TIME), sala (opcional),
status ('marcado' | 'realizado' | 'cancelado'),
created_at, updated_at.

15. Conclusão

O COEPP está maduro em:

Módulos de cadastro (pacientes e estagiários) com validações robustas e segurança.

Lista de prioridade inteligente, que exclui automaticamente pacientes com agendamentos futuros.

Painel com KPIs úteis e um gráfico de atendimentos por dia.

Experiência de uso consistente (máscaras, modais informativos, UI padronizada).

Código limpo, com prepared statements, sanitização e guard de sessão.

Próximos passos sugeridos:

Finalizar/alinhar os endpoints do módulo Agendamentos e Calendário conforme contratos acima.

Implementar CSRF tokens em formulários.

Centralizar JS/CSS e criar componentes reutilizáveis para inputs com máscara/validação.